

Project_3

Soniya Shahi

2024-11-05

```
#the necessary libraries used in the project
library(readxl)
suppressMessages(library(dplyr))
suppressMessages(library(plotly))
library(ggplot2)
library(GGally)
```

Data Preparation

```
#converting the excel data to R dataset
unknown_data <- read_excel("Vole Skulls.xlsm", sheet = "Unknown")
subterraneus_data <- read_excel("Vole Skulls.xlsm", sheet = "Subterraneus")
multiplex_data <- read_excel("Vole Skulls.xlsm", sheet = "Multiplex")
```

```
unknown_data<- unknown_data %>%
rename(
  "Index" = "Vole Skull Index" ,
  "ID" = "Chromosomal Analysis ID",
  "Length" = "Condyllo Incisive Length or Skull Length (0.01mm)",
  "Height" = "Skull Height Above Bullae (0.01mm)",
  "Width" = "Skull Width Across Rostrum (0.01mm)"
)

subterraneus_data<- subterraneus_data %>%
rename(
  "Index" = "Vole Skull Index" ,
  "ID" = "Chromosomal Analysis ID",
  "Length" = "Condyllo Incisive Length or Skull Length (0.01mm)",
  "Height" = "Skull Height Above Bullae (0.01mm)",
  "Width" = "Skull Width Across Rostrum (0.01mm)"
)

multiplex_data<- multiplex_data %>%
rename(
  "Index" = "Vole Skull Index" ,
  "ID" = "Chromosomal Analysis ID",
  "Length" = "Condyllo Incisive Length or Skull Length (0.01mm)",
  "Height" = "Skull Height Above Bullae (0.01mm)",
  "Width" = "Skull Width Across Rostrum (0.01mm)"
)
```

Exploratory Data Analysis

```
#number of rows remaining in unknown_data
raw_remaining_rows <- nrow(unknown_data)
print(raw_remaining_rows)
```

```
## [1] 200
```

Missing values for Subterranean data (Missing values showing in 12th row even though correctly imported)

```
missing_values <- sapply(subterranean_data, function(x) sum(is.na(x)))
# Print the result
print(missing_values)
```

```
## Index      ID Length Height Width
##      1      1      1      1      1
```

Missing values for Multiples data (Missing values showing in 33th row even though correctly imported)

```
missing_values <- sapply(multiplex_data, function(x) sum(is.na(x)))
# Print the result
print(missing_values)
```

```
## Index      ID Length Height Width
##      1      1      1      1      1
```

```
missing_values <- sapply(unknown_data, function(x) sum(is.na(x)))
# Print the result
print(missing_values)
```

```
## Index      ID Length Height Width
##      1      1     41     38     32
```

```
# Check for duplicates in the entire dataframe
duplicates <- duplicated(multiplex_data)
```

```
# Print rows that are duplicates
duplicate_rows <- multiplex_data[duplicates, ]
print(duplicate_rows)
```

```
## # A tibble: 1 x 5
##   Index ID          Length Height Width
##   <dbl> <chr>      <dbl>  <dbl> <dbl>
## 1    29 multiplex    2350    770   456
```

```
# Delete duplicates in the entire dataframe
multiplex_data <- distinct(multiplex_data)
```

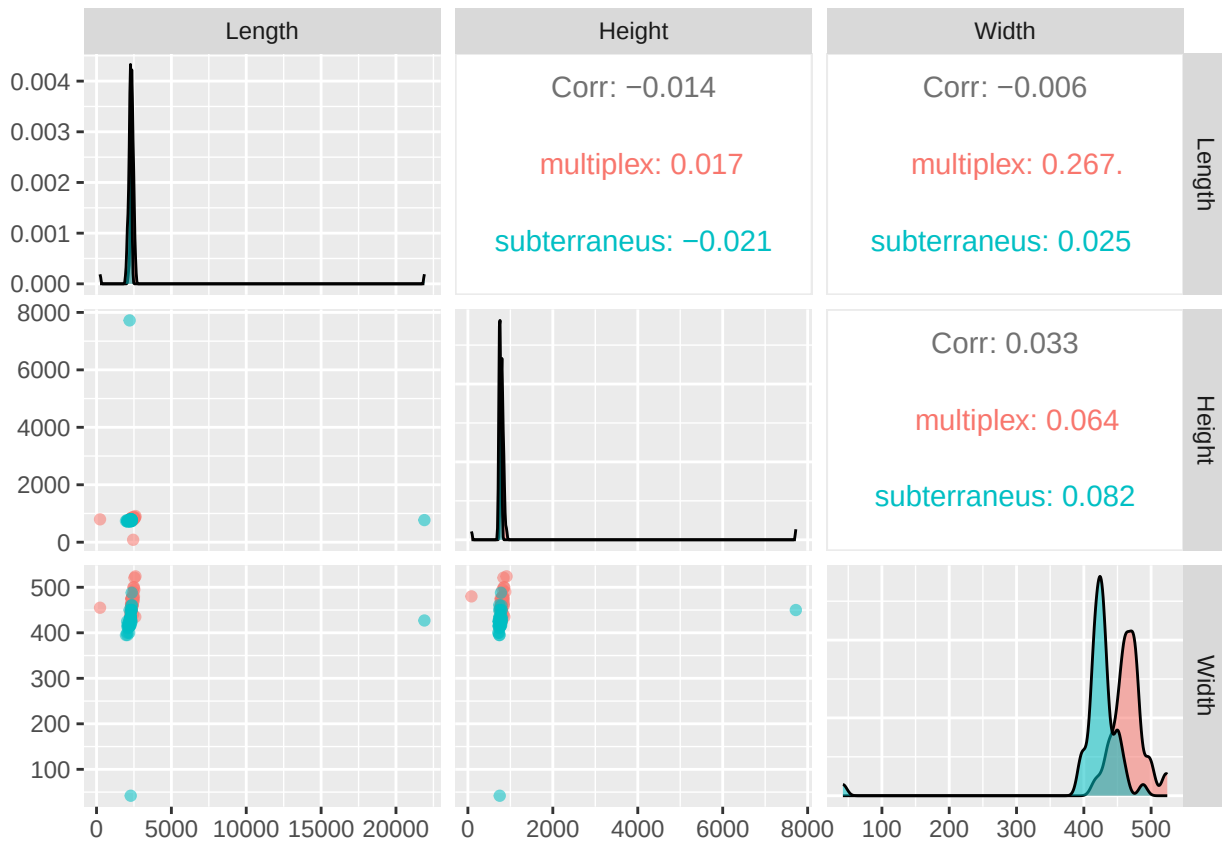
We then omitted the missing values from both the data frame.

```
# Remove rows with missing values
subterraneus_data <- na.omit(subterraneus_data)
multiplex_data <- na.omit(multiplex_data)
```

We then checked outlier for the data frames. Starting with Subterraneus Data.

```
#merge data sets, analysis id, length, height and width
```

```
Orginal_merged_data <- multiplex_data %>% full_join(subterraneus_data, by = c("ID", "Length", "Height"))
ggpairs(Orginal_merged_data, columns = c("Length", "Height", "Width"), mapping = aes(color = ID, alpha = 0.5))
```



```
# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(subterraneus_data, x = ~subterraneus_data$Length, y = ~subterraneus_data$Width, z = ~subterraneus_data$Height)

# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)

# Display the plot
scatter3d
```

Now handling the outliers $\rightarrow$ setting them to the 97.5th percentile value. Upper bound set to 97.5th percentile Lower bound set to 2.5th percentile

```

# Calculate IQR-based lower and upper bounds for x, y, and z
Q1_x <- quantile(subterraneus_data[[3]], 0.25)
Q3_x <- quantile(subterraneus_data[[3]], 0.75)
IQR_x <- Q3_x - Q1_x

Q1_y <- quantile(subterraneus_data[[4]], 0.25)
Q3_y <- quantile(subterraneus_data[[4]], 0.75)
IQR_y <- Q3_y - Q1_y

Q1_z <- quantile(subterraneus_data[[5]], 0.25)
Q3_z <- quantile(subterraneus_data[[5]], 0.75)
IQR_z <- Q3_z - Q1_z

# Replace outliers with median for x
median_x <- median(subterraneus_data[[3]], na.rm = TRUE)
subterraneus_data[[3]][subterraneus_data[[3]] < (Q1_x - 1.5 * IQR_x)] <- median_x
subterraneus_data[[3]][subterraneus_data[[3]] > (Q3_x + 1.5 * IQR_x)] <- median_x

# Replace outliers with median for y
median_y <- median(subterraneus_data[[4]], na.rm = TRUE)
subterraneus_data[[4]][subterraneus_data[[4]] < (Q1_y - 1.5 * IQR_y)] <- median_y
subterraneus_data[[4]][subterraneus_data[[4]] > (Q3_y + 1.5 * IQR_y)] <- median_y

# Replace outliers with median for z
median_z <- median(subterraneus_data[[5]], na.rm = TRUE)
subterraneus_data[[5]][subterraneus_data[[5]] < (Q1_z - 1.5 * IQR_z)] <- median_z
subterraneus_data[[5]][subterraneus_data[[5]] > (Q3_z + 1.5 * IQR_z)] <- median_z

```

Now checking the data distribution after the outlier is manipulated and replaced.

```

# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(subterraneus_data, x = ~subterraneus_data$Length, y = ~subterraneus_data$Width, z = ~subterraneus_data$Height)

# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)

# Display the plot
scatter3d

```

Now we can clearly see that all the data points fall under the same cluster.

We now repeat the same process of finding the outliers and replacing them with upper bound value and

lower bound value for the multiplex dataframe.

```
# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(multiplex_data, x = ~multiplex_data[[3]], y = ~multiplex_data[[4]], z = ~multiplex_data[[5]])

# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)
# Display the plot
scatter3d
```

```
# Calculate the 2.5th and 97.5th percentiles for x, y, and z  
Q1_x <- quantile(multiplex_data[[3]], 0.025)
```

```

Q3_x <- quantile(multiplex_data[[3]], 0.975)
IQR_x <- Q3_x - Q1_x

Q1_y <- quantile(multiplex_data[[4]], 0.025)
Q3_y <- quantile(multiplex_data[[4]], 0.975)
IQR_y <- Q3_y - Q1_y

Q1_z <- quantile(multiplex_data[[5]], 0.025)
Q3_z <- quantile(multiplex_data[[5]], 0.975)
IQR_z <- Q3_z - Q1_z

# Replace outliers with median for x
median_x <- median(multiplex_data[[3]], na.rm = TRUE)
multiplex_data[[3]][multiplex_data[[3]] < (Q1_x - 1.5 * IQR_x)] <- median_x
multiplex_data[[3]][multiplex_data[[3]] > (Q3_x + 1.5 * IQR_x)] <- median_x

# Replace outliers with median for y
median_y <- median(multiplex_data[[4]], na.rm = TRUE)
multiplex_data[[4]][multiplex_data[[4]] < (Q1_y - 1.5 * IQR_y)] <- median_y
multiplex_data[[4]][multiplex_data[[4]] > (Q3_y + 1.5 * IQR_y)] <- median_y

# Replace outliers with median for z
median_z <- median(multiplex_data[[5]], na.rm = TRUE)
multiplex_data[[5]][multiplex_data[[5]] < (Q1_z - 1.5 * IQR_z)] <- median_z
multiplex_data[[5]][multiplex_data[[5]] > (Q3_z + 1.5 * IQR_z)] <- median_z

# Replace outliers with median for z
multiplex_data[[5]][multiplex_data[[5]] < Q1_z] <- median_z
multiplex_data[[5]][multiplex_data[[5]] > Q3_z] <- median_z

# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(multiplex_data, x = ~multiplex_data[[3]], y = ~multiplex_data[[4]], z = ~multiplex_data[[5]])

# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)

# Display the plot
scatter3d

```

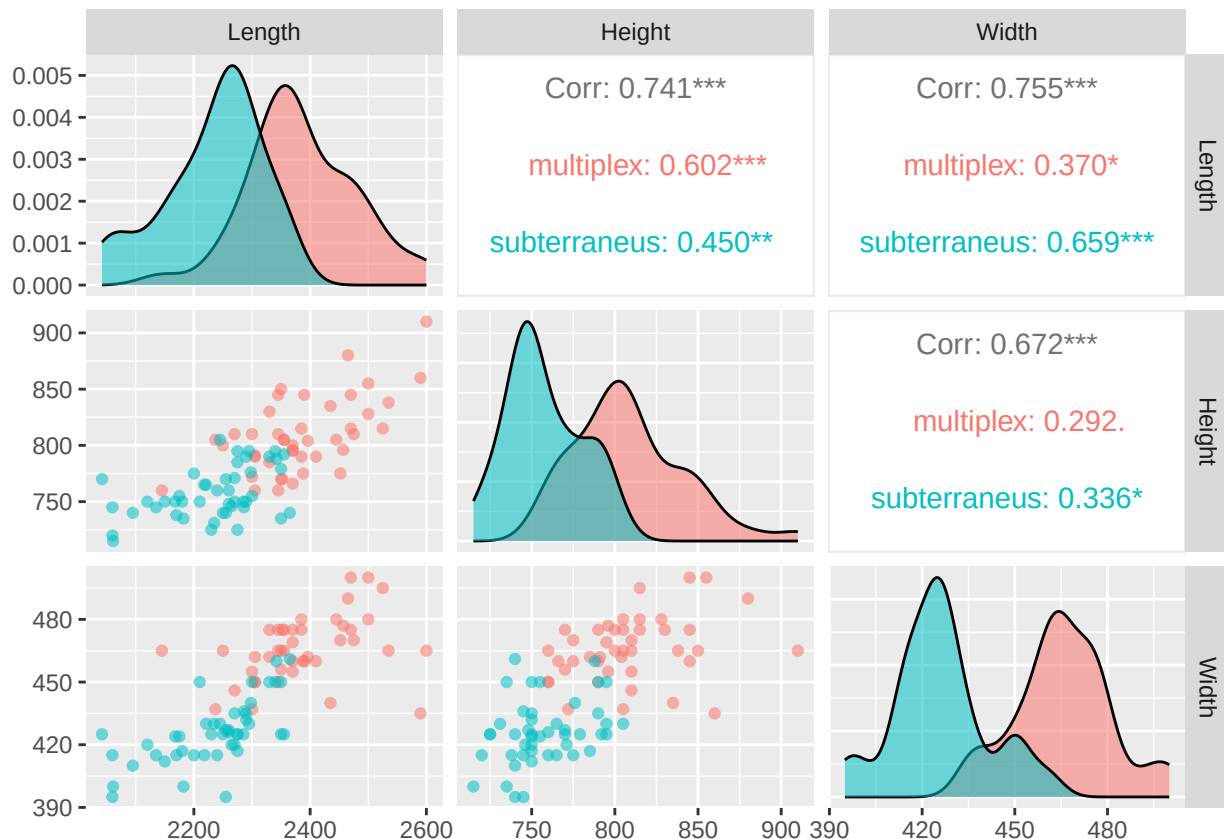
Now we check the data distribution for both Subterraneus and Multiplex data. First we merge them and then check the distribution.

```

# we have two data set known chromosomal analysis and one with unknown
subterraneus_data <- subterraneus_data[,-1]
multiplex_data <- multiplex_data[,-1]
#merge data sets, analysis id, length, height and width
merged_data <- multiplex_data %>% full_join(subterraneus_data, by = c("ID", "Length", "Height", "Width"))

ggpairs(
  merged_data,
  columns = c("Length", "Height", "Width"), # data columns
  mapping = aes(color = ID, alpha = 0.2) # color by chromosomal ID
)

```



Examining the data, we observe scattered values for each variable, with intersections between certain values. Notably, there is overlap between classes.

```

unknown_data <- unknown_data[complete.cases(unknown_data[, "Index"]), ]
unknown_data

```

```

## # A tibble: 199 x 5
##   Index ID      Length Height Width
##   <dbl> <chr>   <dbl>   <dbl> <dbl>
## 1     1 1 unknown  2232    821   430
## 2     2 2 unknown    NA    755   405
## 3     3 3 unknown  2295    NA    NA
## 4     4 4 unknown  2355    842   490

```

```
## 5      5 unknown  2335    814   481
## 6      6 unknown  2355     NA   460
## 7      7 unknown  2416    850  5000
## 8      8 unknown  2423    760    NA
## 9      9 unknown  2504    835   487
## 10     10 unknown  2100    720    40
## # i 189 more rows
```

We will be using performing cross-validation for multiple linear regression models to assess their performance.

```
# Set seed for reproducibility
set.seed(123)

# Filter rows without missing values
complete_data <- na.omit(unknown_data)

# Define the number of folds for cross-validation
num_folds <- 5

# Create a data frame to store cross-validation results
cv_results <- data.frame(Model = character(), R_squared = numeric(), MAPE = numeric(), stringsAsFactors = FALSE)

# List of models
models <- list(
  "Width from Length and Height" = "Width ~ Length + Height",
  "Width from Length" = "Width ~ Length",
  "Width from Height" = "Width ~ Height",
  "Height from Length and Width" = "Height ~ Length + Width",
  "Height from Length" = "Height ~ Length",
  "Height from Width" = "Height ~ Width"
)

# Perform k-fold cross-validation for each model
for (model_name in names(models)) {
  formula <- as.formula(models[[model_name]])

  # Create a model using lm
  model <- lm(formula, data = complete_data)

  # Access R-squared value from the summary of the linear model
  r_squared <- sprintf("%.3f", summary(model)$r.squared)

  # Make predictions on the entire dataset
  predictions <- predict(model, newdata = complete_data)

  # Extract response variable using model.frame
  response_variable <- all.vars(formula)[1]

  # Calculate MAPE with three decimal places
  mape <- sprintf("%.3f", mean(abs((predictions - complete_data[[response_variable]])) / complete_data[[response_variable]]))

  # Append results to the data frame
  cv_results <- rbind(cv_results, data.frame(Model = model_name, R_squared = as.numeric(r_squared), MAPE = as.numeric(mape)))
}
```

```

}

# Print the cross-validation results
print(cv_results)

```

```

##              Model R_squared  MAPE
## 1 Width from Length and Height    0.037 18.624
## 2              Width from Length    0.000 19.942
## 3              Width from Height    0.036 18.655
## 4 Height from Length and Width    0.048  4.814
## 5              Height from Length    0.012  4.901
## 6              Height from Width    0.036  4.840

```

Now we merge the unknown data with all the data.

```

merged_data <- merged_data %>% full_join(unknown_data, by = c("ID", "Length", "Height", "Width" ))

```

We repeat the similar process to check for the outlier and replace it with upper bound and lower bound.

```

# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(merged_data, x = ~merged_data[[2]], y = ~merged_data[[3]], z = ~merged_data[[4]],

# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)
# Display the plot
scatter3d

```

```
# Calculate the 2.5th and 97.5th percentiles for x, y, and z  
Q1_x <- quantile(merged_data[[2]], 0.025, na.rm = TRUE)
```

```

Q3_x <- quantile(merged_data[[2]], 0.975, na.rm = TRUE)
IQR_x <- Q3_x - Q1_x

Q1_y <- quantile(merged_data[[3]], 0.025, na.rm = TRUE)
Q3_y <- quantile(merged_data[[3]], 0.975, na.rm = TRUE)
IQR_y <- Q3_y - Q1_y

Q1_z <- quantile(merged_data[[4]], 0.025, na.rm = TRUE)
Q3_z <- quantile(merged_data[[4]], 0.975, na.rm = TRUE)
IQR_z <- Q3_z - Q1_z

# Replace outliers with median for x
median_x <- median(merged_data[[2]], na.rm = TRUE)
merged_data[[2]][merged_data[[2]] < (Q1_x - 1.5 * IQR_x)] <- median_x
merged_data[[2]][merged_data[[2]] > (Q3_x + 1.5 * IQR_x)] <- median_x

# Replace outliers with median for y
median_y <- median(merged_data[[3]], na.rm = TRUE)
merged_data[[3]][merged_data[[3]] < (Q1_y - 1.5 * IQR_y)] <- median_y
merged_data[[3]][merged_data[[3]] > (Q3_y + 1.5 * IQR_y)] <- median_y

# Replace outliers with median for z
median_z <- median(merged_data[[4]], na.rm = TRUE)
merged_data[[4]][merged_data[[4]] < (Q1_z - 1.5 * IQR_z)] <- median_z
merged_data[[4]][merged_data[[4]] > (Q3_z + 1.5 * IQR_z)] <- median_z

# Create 3D scatter plot with specified columns
scatter3d <- plot_ly(merged_data, x = ~merged_data[[2]], y = ~merged_data[[3]], z = ~merged_data[[4]],

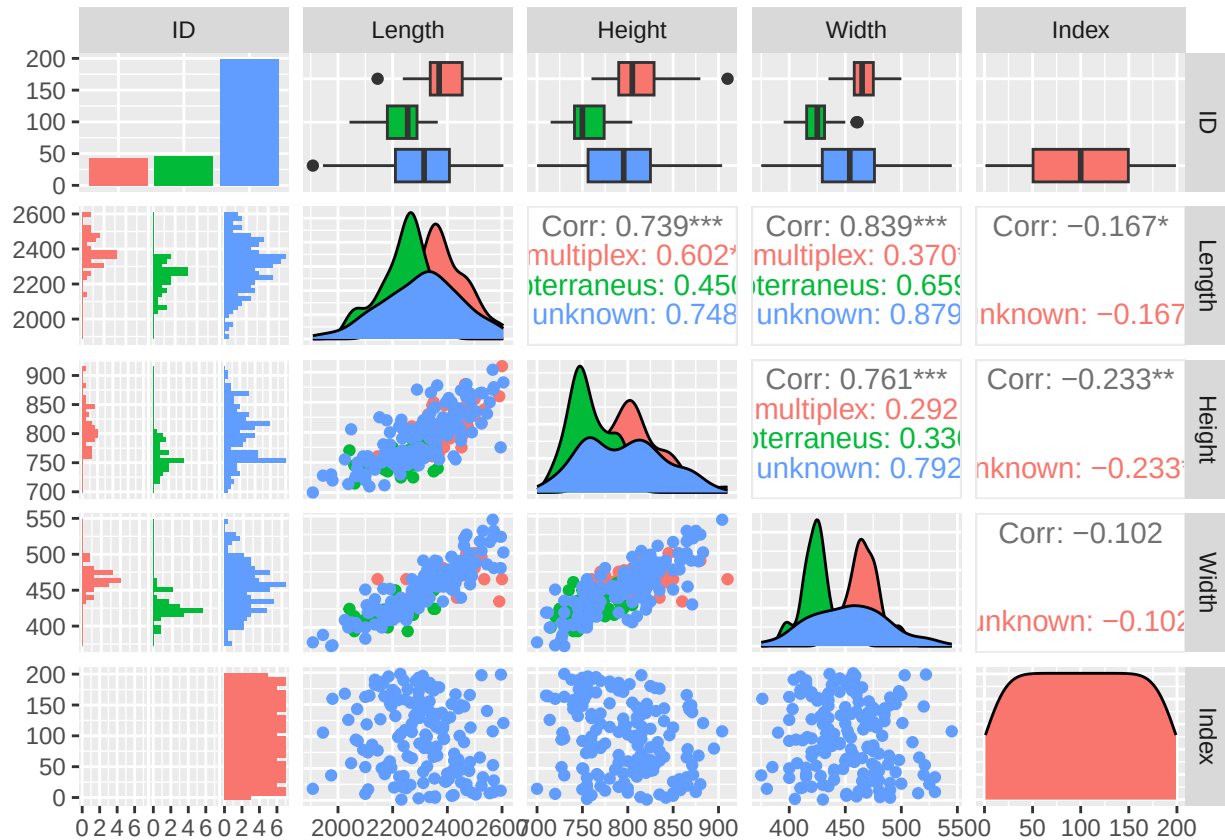
# Customize layout directly in the plot_ly function
scatter3d <- scatter3d %>% layout(
  scene = list(
    xaxis = list(title = 'Skull Length'),
    yaxis = list(title = 'Skull Width'),
    zaxis = list(title = 'Skull Height')
  )
)

# Display the plot
scatter3d

```



```
ggpairs(merged_data, ggplot2::aes(colour=~ID))
```



Now, from the above graph we can clearly see the unknown data has a significant cluster in between the overlapping intersection between multiplex and subterranean data. Hence, that niche area will be very difficult to predict.

Now we use cross validation with 5 fold as a function to train and test the model.

```
perform_cross_validation <- function(data, formula) {
  # Convert 'ID' to a factor
  data$ID <- as.factor(data$ID)

  # Filter data for training and cross-validation (excluding 'unknown')
  filtered_data <- data[data$ID != "unknown", ]

  # Set a seed for reproducibility
  set.seed(123)

  # Split data into 5 folds
  folds <- cut(seq(1, nrow(filtered_data)), breaks = 5, labels = FALSE)

  # Initialize vectors to store results
  train_accuracies <- numeric(length = 5)
  valid_accuracies <- numeric(length = 5)

  # Perform 5-fold cross-validation
```

```

for (i in 1:5) {
  # Create indices for training and validation sets
  train_indices <- which(folds != i)
  valid_indices <- which(folds == i)

  # Fit the model on the training set
  model <- glm(formula, data = filtered_data[train_indices, ], family = "binomial")

  # Make predictions on the training set
  train_predictions <- predict(model, newdata = filtered_data[train_indices, ], type = "response") > 0.5
  train_predictions <- ifelse(train_predictions, "subterraneus", "multiplex")

  # Make predictions on the validation set
  valid_predictions <- predict(model, newdata = filtered_data[valid_indices, ], type = "response") > 0.5
  valid_predictions <- ifelse(valid_predictions, "subterraneus", "multiplex")

  # Calculate and store accuracies
  train_accuracies[i] <- mean(train_predictions == filtered_data$ID[train_indices])
  valid_accuracies[i] <- mean(valid_predictions == filtered_data$ID[valid_indices])
}

# Calculate mean accuracies
mean_train_accuracy <- mean(train_accuracies)
mean_valid_accuracy <- mean(valid_accuracies)

# Return the results
return(list(
  train_accuracies = train_accuracies,
  valid_accuracies = valid_accuracies,
  mean_train_accuracy = mean_train_accuracy,
  mean_valid_accuracy = mean_valid_accuracy
))
}

```

The following chunk will takes in merged data train and test the 7 model Length + Width + Height Length + Width Length + Height Width + Height Length Width Height

```

# Convert 'ID' to a factor
merged_data$ID <- as.factor(merged_data$ID)

# Define a vector of formulas
formulas <- c(
  "ID ~ Length + Width + Height",
  "ID ~ Length + Width",
  "ID ~ Length + Height",
  "ID ~ Width + Height",
  "ID ~ Length",
  "ID ~ Width",
  "ID ~ Height"
)

# Initialize a dataframe to store the results
results_df <- data.frame(Formula = character(), Mean_Training_Accuracy = numeric(), Mean_Validation_Accuracy = numeric())

```

```

# Perform cross-validation for each formula
for (formula_text in formulas) {
  formula <- as.formula(formula_text)

  # Perform cross-validation using the function
  results <- perform_cross_validation(merged_data, formula)

  # Add results to the dataframe
  results_df <- rbind(results_df, data.frame(Formula = formula_text, Mean_Training_Accuracy = results$Mean_Training_Accuracy, Mean_Validation_Accuracy = results$Mean_Validation_Accuracy))
}

# Print the results dataframe
print(results_df)

```

	Formula	Mean_Training_Accuracy	Mean_Validation_Accuracy
## 1	ID ~ Length + Width + Height	0.9269562	0.8986928
## 2	ID ~ Length + Width	0.9044601	0.8535948
## 3	ID ~ Length + Height	0.8511346	0.7653595
## 4	ID ~ Width + Height	0.9213224	0.9098039
## 5	ID ~ Length	0.8510563	0.7751634
## 6	ID ~ Width	0.8932707	0.8313725
## 7	ID ~ Height	0.8061424	0.7522876

The model featuring only Width and Height exhibits the highest training and validation accuracy relative to others. Its interpretation is more straightforward compared to models with additional features. This observation suggests that the inclusion of more features does not necessarily guarantee an improvement in model accuracy.

Stepwise Regression Model

```

# Assuming 'merged_data' is our merged dataset
library(boot)

# Convert 'ID' to a factor
merged_data$ID <- as.factor(merged_data$ID)

# Filter data for training (excluding 'unknown')
train_data <- merged_data[merged_data$ID != "unknown", ]

# Specify the formula for the logistic regression model
formula <- ID ~ Length + Height + Width + Length * Width + Length * Height + Width * Height

# Fit the stepwise logistic regression model using only training data
stepwise_model <- step(glm(formula, data = train_data, family = "binomial"), direction = "both")

```

```

## Start:  AIC=44.45
## ID ~ Length + Height + Width + Length * Width + Length * Height +
##       Width * Height
##
##           Df Deviance    AIC
## - Length:Height  1   30.454 42.454
## - Height:Width   1   30.467 42.467

```

```

## - Length:Width 1 30.608 42.609
## <none> 30.451 44.451
##
## Step: AIC=42.45
## ID ~ Length + Height + Width + Length:Width + Height:Width
##
## Df Deviance AIC
## - Height:Width 1 30.468 40.468
## - Length:Width 1 30.641 40.641
## <none> 30.454 42.454
## + Length:Height 1 30.451 44.451
##
## Step: AIC=40.47
## ID ~ Length + Height + Width + Length:Width
##
## Df Deviance AIC
## - Length:Width 1 30.642 38.642
## <none> 30.468 40.468
## + Height:Width 1 30.454 42.454
## + Length:Height 1 30.467 42.467
## - Height 1 39.558 47.558
##
## Step: AIC=38.64
## ID ~ Length + Height + Width
##
## Df Deviance AIC
## - Length 1 30.642 36.642
## <none> 30.642 38.642
## + Length:Width 1 30.468 40.468
## + Length:Height 1 30.609 40.609
## + Height:Width 1 30.641 40.641
## - Height 1 40.334 46.334
## - Width 1 55.594 61.594
##
## Step: AIC=36.64
## ID ~ Height + Width
##
## Df Deviance AIC
## <none> 30.642 36.642
## + Height:Width 1 30.641 38.641
## + Length 1 30.642 38.642
## - Height 1 44.068 48.068
## - Width 1 67.266 71.266

# Display the final summary for the stepwise model
summary(stepwise_model)

##
## Call:
## glm(formula = ID ~ Height + Width, family = "binomial", data = train_data)
##
## Coefficients:
## Estimate Std. Error z value Pr(>|z|)
## (Intercept) 114.88659 28.21712 4.072 4.67e-05 ***

```

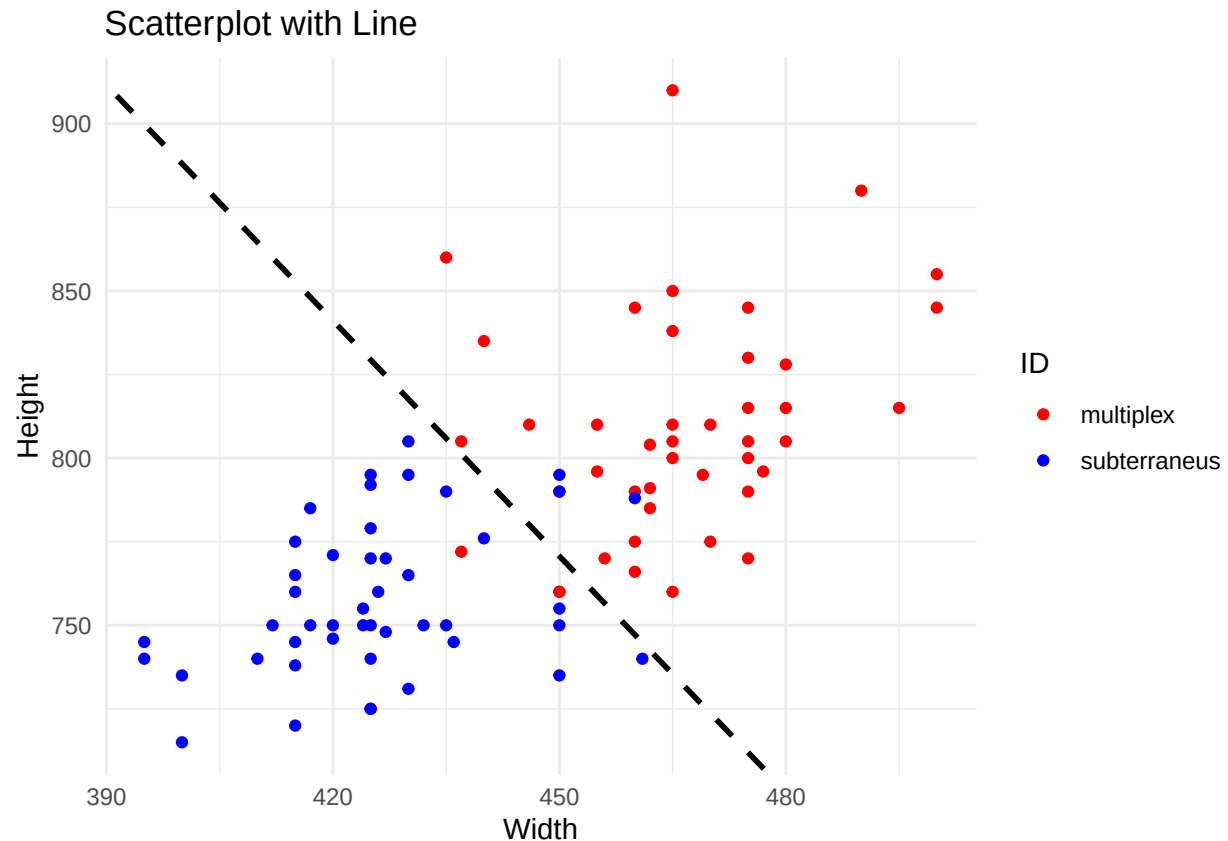
```
## Height      -0.05755    0.02040  -2.820 0.004798 **
## Width       -0.15659    0.04178  -3.748 0.000178 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 123.279  on 88  degrees of freedom
## Residual deviance:  30.642  on 86  degrees of freedom
## AIC: 36.642
##
## Number of Fisher Scoring iterations: 7
```

The provided output illustrates a stepwise logistic regression process with the goal of minimizing the Akaike Information Criterion (AIC) to find the most parsimonious model for predicting the variable “ID.” The stepwise selection starts with a model including interactions between Length, Height, and Width. At each step, the algorithm evaluates the impact of removing or adding terms to the model based on AIC. The final selected model is a logistic regression model with the formula “ID ~ Height + Width.” The estimated coefficients indicate that both Height and Width are statistically significant predictors of the response variable “ID,” with negative coefficients suggesting a negative impact on the log-odds of the outcome. The resulting model has a residual deviance of 48.086 on 87 degrees of freedom, and its AIC is 54.086. This stepwise process highlights a streamlined model, emphasizing the significance of Height and Width in predicting the binary outcome “ID.”

```
# Convert 'ID' to a factor
merged_data$ID <- as.factor(merged_data$ID)

# Coefficients of the line (replace with your actual coefficients)
intercept <- -77.71775/(-0.04253)
slope <- -(-0.09987) / (-0.04253)

# Create a scatterplot using ggplot2
ggplot(train_data, aes(x = Width, y = Height, color = ID)) +
  geom_point() +
  geom_abline(intercept = intercept, slope = slope, color = "black", linetype = "dashed", size = 1) +
  labs(title = "Scatterplot with Line",
       x = "Width",
       y = "Height") +
  scale_color_manual(values = c("subterranean" = "blue", "multiplex" = "red")) +
  theme_minimal() +
  coord_cartesian(xlim = c(min(train_data$Width), max(train_data$Width)),
                  ylim = c(min(train_data$Height), max(train_data$Height)))
```



The provided figure illustrates a decision boundary, where data points located above the line are classified as multiples, and those below are classified as subterraneus. Additionally, it visually conveys that predictions are not consistently accurate, as some points deviate from their actual classifications. This is for the training dataset.

```
# Filter out rows where ID is not "unknown"
training_data <- merged_data[merged_data$ID != "unknown", ]

# Convert 'ID' to a factor
training_data$ID <- as.factor(training_data$ID)

# Initialize a list to store the models
models <- list()

# Train models for each formula
for (formula_text in formulas) {
  formula <- as.formula(formula_text)

  # Train the model using the training_data
  model <- glm(formula, data = training_data, family = binomial())

  # Store the model in the list
  models[[formula_text]] <- model
}

# Function to convert probabilities to class labels
```

```

convert_to_class <- function(probabilities, threshold = 0.5) {
  return(ifelse(probabilities > threshold, "subterraneus", "multiplex"))
}

```

```

predict_from_datasets <- function(dataset, models) {
  # Omit datasets with missing values in all three columns "Length", "Width", and "Height"
  dataset <- dataset[!(is.na(dataset$Length) & is.na(dataset$Width) & is.na(dataset$Height)), ]

  # Split the dataset based on missing values
  dataset1 <- dataset[(is.na(dataset$Length) & !(is.na(dataset$Width)) & !(is.na(dataset$Height))) | complete.cases(dataset[c("Length", "Height")]), ]
  dataset2 <- dataset[is.na(dataset$Width) & complete.cases(dataset[c("Length", "Height")]), ]
  dataset3 <- dataset[is.na(dataset$Height) & complete.cases(dataset[c("Length", "Width")]), ]
  dataset4 <- dataset[is.na(dataset$Length) & is.na(dataset$Height) & !(is.na(dataset$Width)), ]
  dataset5 <- dataset[is.na(dataset$Height) & is.na(dataset$Width) & !(is.na(dataset$Length)), ]
  dataset6 <- dataset[is.na(dataset$Length) & is.na(dataset$Width) & !(is.na(dataset$Height)), ]

  # Make predictions and store in respective dataset's predicted_ID column
  dataset1$predicted_ID <- convert_to_class(predict(models[["ID ~ Width + Height"]], newdata = dataset1)
  dataset2$predicted_ID <- convert_to_class(predict(models[["ID ~ Length + Height"]], newdata = dataset2)
  dataset3$predicted_ID <- convert_to_class(predict(models[["ID ~ Length + Width"]], newdata = dataset3)
  dataset4$predicted_ID <- convert_to_class(predict(models[["ID ~ Width"]], newdata = dataset4, type = "class"))
  dataset5$predicted_ID <- convert_to_class(predict(models[["ID ~ Length"]], newdata = dataset5, type = "class"))
  dataset6$predicted_ID <- convert_to_class(predict(models[["ID ~ Height"]], newdata = dataset6, type = "class"))

  # Print the length of each dataset and the corresponding model used
  print(paste("No. of Data:", nrow(dataset1), ", Model Used: ID ~ Width + Height"))
  print(paste("  Subterraneus Count:", sum(dataset1$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset1$predicted_ID == "multiplex")))

  print(paste("No. of Data:", nrow(dataset2), ", Model Used: ID ~ Length + Height"))
  print(paste("  Subterraneus Count:", sum(dataset2$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset2$predicted_ID == "multiplex")))

  print(paste("No. of Data:", nrow(dataset3), ", Model Used: ID ~ Length + Width"))
  print(paste("  Subterraneus Count:", sum(dataset3$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset3$predicted_ID == "multiplex")))

  print(paste("No. of Data:", nrow(dataset4), ", Model Used: ID ~ Width"))
  print(paste("  Subterraneus Count:", sum(dataset4$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset4$predicted_ID == "multiplex")))

  print(paste("No. of Data:", nrow(dataset5), ", Model Used: ID ~ Length"))
  print(paste("  Subterraneus Count:", sum(dataset5$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset5$predicted_ID == "multiplex")))

  print(paste("No. of Data:", nrow(dataset6), ", Model Used: ID ~ Height"))
  print(paste("  Subterraneus Count:", sum(dataset6$predicted_ID == "subterraneus")))
  print(paste("  Microtus Count:", sum(dataset6$predicted_ID == "multiplex")))

  # Merge all datasets and sort by Index
  merged_all <- rbind(dataset1, dataset2, dataset3, dataset4, dataset5, dataset6)
  merged_all <- merged_all[order(merged_all$Index), ]
}

```

```

# Return the merged dataset
return(merged_all)
}

```

```
unknown_data <- predict_from_datasets(unknown_data, models)
```

```

## [1] "No. of Data: 135 , Model Used: ID ~ Width + Height"
## [1] "   Subterraneus Count: 56"
## [1] "   Microtus Count: 79"
## [1] "No. of Data: 18 , Model Used: ID ~ Length + Height"
## [1] "   Subterraneus Count: 8"
## [1] "   Microtus Count: 10"
## [1] "No. of Data: 20 , Model Used: ID ~ Length + Width"
## [1] "   Subterraneus Count: 10"
## [1] "   Microtus Count: 10"
## [1] "No. of Data: 13 , Model Used: ID ~ Width"
## [1] "   Subterraneus Count: 4"
## [1] "   Microtus Count: 9"
## [1] "No. of Data: 4 , Model Used: ID ~ Length"
## [1] "   Subterraneus Count: 2"
## [1] "   Microtus Count: 2"
## [1] "No. of Data: 9 , Model Used: ID ~ Height"
## [1] "   Subterraneus Count: 3"
## [1] "   Microtus Count: 6"

```

```
unknown_data
```

```

## # A tibble: 199 x 6
##   Index ID      Length Height Width predicted_ID
##   <dbl> <chr>    <dbl> <dbl> <dbl> <chr>
## 1     1 1 unknown  2232   821   430 subterraneus
## 2     2 2 unknown    NA   755   405 subterraneus
## 3     3 3 unknown  2295    NA    NA subterraneus
## 4     4 4 unknown  2355   842   490 multiplex
## 5     5 5 unknown  2335   814   481 multiplex
## 6     6 6 unknown  2355    NA   460 multiplex
## 7     7 7 unknown  2416   850  5000 multiplex
## 8     8 8 unknown  2423   760    NA multiplex
## 9     9 9 unknown  2504   835   487 multiplex
## 10    10 10 unknown  2100   720    40 subterraneus
## # i 189 more rows

```

```

# Assuming 'merged_dataset' is your dataset
predicted_id_counts <- table(unknown_data$predicted_ID)

# Print the counts
print("Predicted_ID Counts:")

```

```
## [1] "Predicted_ID Counts:"
```

```
print(predicted_id_counts)
```

```
##  
##    multiplex subterraneus  
##          116           83
```